

ICP133 - Fundamentos de Sistemas de Computação

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
May 19, 2026

Aula 7 - Circuitos e Portas Lógicas

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
May 19, 2026

- **Sistema Binário:** Utiliza apenas dois dígitos (1 e 0), representados por faixas de tensão, ideais para representar relações lógicas.
- **Lógica Booleana:** Desenvolvida por George Boole (1854), baseia-se na tomada de decisões lógicas sobre circunstâncias verdadeiras ou falsas.
- **Álgebra Booleana:** Sistema que emprega símbolos (como A, B e C) e operadores para representar expressões lógicas, simplificando a notação.

- **O que são Estados Lógicos?**
 - Representações de condições que assumem apenas **dois valores possíveis**: ou é uma coisa, ou é outra.
- **Exemplos do cotidiano:**
 - Lâmpada: Acesa *ou* Apagada
 - Condição: Vivo *ou* Morto
 - Evento: Aconteceu *ou* Não aconteceu
 - Existência: Existe *ou* Não existe
- **Tradução Tecnológica:**
 - **Álgebra Booleana:** Traduz situações para valores Verdadeiro (*True*) ou Falso (*False*).
 - **Eletrônica Digital:** Lê estados como níveis de tensão (Binário 1 e Binário 0).

Álgebra Booleana vs. Convencional

- **Diferença Principal:** Na álgebra booleana, constantes e variáveis podem ter apenas dois valores possíveis: **0** ou **1**.
- **Natureza das Variáveis:** Não representam números efetivos, mas o estado do **nível lógico** de uma tensão.
- **Simplicidade:** Não existem frações, decimais, números negativos, logaritmos ou números imaginários.

- **Definição:** Modo de expressar a relação entre as entradas e as saídas de um circuito lógico.
- **Representação:** Usamos letras (ex: A, B, C) para representar as variáveis.
- **Dualidade:** Em um instante qualquer, $A = 0$ ou $A = 1$. Se não for um valor, será obrigatoriamente o outro.

Níveis de Tensão e Faixas de Operação

Em um sistema digital, o valor lógico é determinado pela tensão medida:

- **Nível Lógico 0:** Qualquer tensão entre 0 e 0,8 V.
- **Nível Lógico 1:** Qualquer tensão entre 2 e 5 V.

Observação Importante

Tensões entre 0,8 V e 2 V são consideradas **indefinidas** e não devem ocorrer em circunstâncias normais de operação.

Termos Sinônimos para Níveis Lógicos

Os níveis 0 e 1 são referenciados por diversos termos (Tabela 3.1):

Nível Lógico 0	Nível Lógico 1
Falso	Verdadeiro
Desativado (OFF)	Ativado (ON)
Baixo (LOW)	Alto (HIGH)
Não	Sim
Aberto	Fechado
Apagado	Aceso
Morto	Vivo
Não aconteceu	Aconteceu
Inexistente	Existente

Operações Lógicas Básicas

A álgebra booleana possui apenas três operações fundamentais:

1. **OR (OU)**
2. **AND (E)**
3. **NOT (NÃO)**

Portas Lógicas

Circuitos digitais (com diodos, transistores e resistores) que realizam essas operações básicas sobre as entradas para gerar uma saída.

- **Portas Lógicas:** São os blocos fundamentais de todos os circuitos e sistemas digitais.
- **Análise e Simplificação:** A álgebra permite reescrever circuitos utilizando menos portas e conexões.
- **Síntese:** Atua como ferramenta de projeto para criar relações específicas entre entrada e saída.

Finalidades da Álgebra Booleana

A álgebra booleana é utilizada para dois objetivos principais:

- **Descrição e Análise:** Entender o funcionamento individual das portas lógicas básicas.
- **Projeto:** Analisar e projetar circuitos combinacionais complexos a partir da união dessas portas.

Tabelas-Verdade: Conceito

- **Definição:** Técnica para descrever como a saída de um circuito lógico depende dos níveis lógicos presentes em suas entradas.
- **Estrutura:**
 - **Lado Esquerdo:** Lista todas as combinações possíveis das entradas ($A, B, C \dots$).
 - **Lado Direito:** Indica o nível lógico resultante na saída (x) para cada combinação.
- **Funcionamento:** Cada linha representa um estado específico do sistema. Se as entradas mudam, a saída segue o valor definido na linha correspondente.

Tabelas Verdade

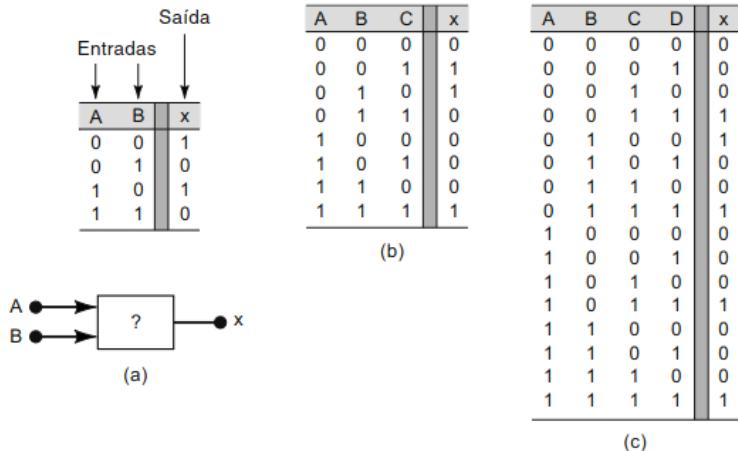


FIGURA 3.1 Exemplos de tabelas-verdade para circuitos de (a) duas, (b) três e (c) quatro entradas.

Número de Combinações

O número de linhas de uma tabela-verdade depende da quantidade de entradas (N):

Regra Exponencial

O número de combinações de entrada é igual a 2^N .

- **2 entradas:** $2^2 = 4$ combinações (linhas).
- **3 entradas:** $2^3 = 8$ combinações (linhas).
- **4 entradas:** $2^4 = 16$ combinações (linhas).

- **Preenchimento:** Para não esquecer nenhuma combinação, as entradas são listadas em **sequência de contagem binária**.
- **Exemplo (2 entradas):**
 - 00 (zero)
 - 01 (um)
 - 10 (dois)
 - 11 (três)

Questões para Revisão

1. Qual será o estado lógico da saída para o circuito de quatro entradas [Figura 3.1(c)] quando todas as entradas, exceto a B, forem nível 1?
2. Qual o estado da saída para as condições: $A = 1$, $B = 0$, $C = 1$ e $D = 0$?
3. Quantas linhas deve ter uma tabela que representa um circuito de cinco entradas?

Operação OR (OU)

- **Definição:** A primeira das três operações booleanas básicas.
- **Analogia Prática (Forno):**
 - A lâmpada acende ($x = 1$) se o interruptor for acionado ($A = 1$) **OU** se a porta for aberta ($B = 1$).
- **Regra de Saída:** A saída x será nível lógico 1 se uma ou mais entradas forem 1.
- **Caso Único de Nível 0:** x só é 0 quando **ambas** as entradas são 0.

- **Equação:** $x = A + B$
- **O Sinal '+':** Não representa a adição convencional; representa a operação OR.
- **Diferença da Álgebra Comum:**
 - Na lógica OR: $1 + 1 = 1$.
 - Como 1 significa nível alto, o resultado nunca é maior que 1.
- **Leitura:** “ x é igual a A ou B ”.

Múltiplas Entradas na Operação OR

A lógica se estende para qualquer número de entradas:

- **Três entradas:** $x = A + B + C$
- **Condição de saída 1:** x será 1 quando A , B , C ou qualquer combinação delas for 1.
- **Exemplo de nível alto:** Se todas as entradas forem 1, teremos:

$$x = 1 + 1 + 1 = 1$$

- **Linguagem Normal:** x é verdadeiro (1) quando A é verdadeiro (1) OU B é verdadeiro (1) OU C é verdadeiro (1).

Tabela-Verdade da Operação OR

Resumo das combinações para duas entradas (Figura 3.2):

A	B	$x = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

OU com 2 entradas



A	B	C	$x = A + B + C$
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

FIGURA 3.3 Símbolo e tabela-verdade para uma porta OR de três entradas.

- **Definição:** Circuito com duas ou mais entradas cuja saída é o resultado da combinação das entradas via operação OR.
- **Níveis Lógicos:** As entradas (A , B) e a saída (x) são representadas por níveis lógicos de tensão.
- **Símbolo Lógico:** Possui representação específica para indicar a operação $x = A + B$.

- **Saída ALTA (1):** Ocorre se a entrada A, ou a entrada B, ou ambas forem nível lógico 1.
- **Saída BAIXA (0):** Ocorre apenas se **todas** as entradas forem nível lógico 0.
- **Resumo:** Basta que uma única entrada esteja em nível 1 para que a saída também seja 1.

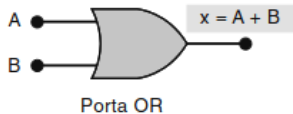
- **Extensibilidade:** O princípio aplica-se a portas OR com qualquer número de entradas (ex: 3 entradas).
- **Expressão Booleana:** Para três entradas, a saída é expressa como $x = A + B + C$.
- **Notação:** O sinal $+$ reforça o uso da operação lógica OR entre as várias entradas.
- **Tabela-Verdade:** Confirma que a saída será 1 em todos os casos onde houver uma ou mais entradas em 1.

OU com 3 entradas

OR

A	B	$x = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

(a)



(b)

FIGURA 3.2 (a) Tabela-verdade que define a operação OR; (b) símbolo de uma porta OR de duas entradas.

- **Regra Fundamental:** A operação OR gera um resultado (saída) **1** sempre que quaisquer das entradas for **1**. Caso contrário, a saída é **0**.
- **Definição de Porta OR:** É o circuito lógico que realiza fisicamente a operação OR sobre as entradas aplicadas.
- **Leitura da Expressão:** A equação $x = A + B$ deve ser lida como: *“x é igual a A ou B”*.

Exemplo Prático: Operação OR

Frase: "Vou à praia se for Sábado (A) OU se estiver Ensolarado (B)"

- **A = 1:** É sábado — **A = 0:** Não é sábado
- **B = 1:** Tem sol — **B = 0:** Não tem sol

A (Sábado)	B (Sol)	x (Praia)
0	0	0
0	1	1
1	0	1
1	1	1

Exemplo Prático: “Eu vou ao parque se for Sábado (A) OU se tiver Sol (B) OU se eu ganhar uma carona (C).”

- **Entradas (Variáveis):**
 - A: É sábado (1 = sim, 0 = não)
 - B: Está sol (1 = sim, 0 = não)
 - C: Ganhei carona (1 = sim, 0 = não)
- **Expressão Booleana:** $x = A + B + C$
- **Lógica:** A saída x é 1 se A ou B ou C (ou qualquer combinação deles) for 1.

Tabela-Verdade: 3 Entradas ($2^3 = 8$ linhas)

A (Sábado)	B (Sol)	C (Carona)	x (Parque)
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

A saída é 0 apenas quando todas as entradas são 0 simultaneamente.

- **Símbolo:** O símbolo possui três terminais de entrada e um de saída.
- **Funcionamento:** Opera de modo que a saída será ALTA (1) se qualquer entrada for ALTA.
- **Soma Booleana:** $1 + 1 + 1 = 1$. O resultado nunca ultrapassa o nível lógico 1.

Operação AND (E)

- **Regra de Saída:** A saída x será nível lógico 1 **apenas** quando todas as entradas forem 1.
- **Caso de Nível 0:** Se qualquer uma das entradas for 0, a saída será obrigatoriamente 0.

- **Equação:** $x = A \cdot B$ ou simplesmente $x = AB$.
- **O Sinal ($\cdot$):** Representa a operação booleana AND.
- **Relação com a Multiplicação:**
 - A operação AND equivale à multiplicação convencional entre variáveis booleanas ($0 \cdot 0 = 0$; $0 \cdot 1 = 0$; $1 \cdot 1 = 1$).
- **Leitura:** “ x é igual a A e B ”.

Múltiplas Entradas na Operação AND

- **Três entradas:** $x = A \cdot B \cdot C = ABC$.
- **Leitura:** “ x é igual a A e B e C ”.
- **Condição de saída 1:** x será 1 **apenas** quando as variáveis A , B e C forem, simultaneamente, nível 1.
- **Exemplo:** Se qualquer entrada for 0, o produto lógico resulta em 0.

Tabela-Verdade da Operação AND

Resumo das combinações para duas entradas:

A	B	$x = AB$
0	0	0
0	1	0
1	0	0
1	1	1

E com 2 entradas

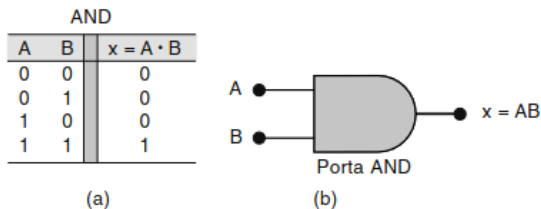


FIGURA 3.7 (a) Tabela-verdade para a operação AND; (b) símbolo da porta AND.

A Porta AND em Circuitos Digitais

- **Definição:** Circuito que opera de modo que sua saída seja nível ALTO **somente** quando todas as entradas também o forem.
- **Expressão de Saída:** A saída é igual ao produto lógico das entradas: $x = AB$.
- **Estados de Operação:**
 - Para todos os outros casos (onde houver pelo menos um zero), a saída será nível BAIXO (0).

- **Extensibilidade:** O princípio é o mesmo para qualquer número de entradas.
- **Exemplos:**
 - Porta de 3 entradas: $x = ABC$.
 - Porta de 4 entradas: $x = ABCD$.
- **Consistência:** A saída será 1 apenas no caso em que todas as entradas ($A = B = C = \dots = 1$) sejam 1.

E com 3 entradas

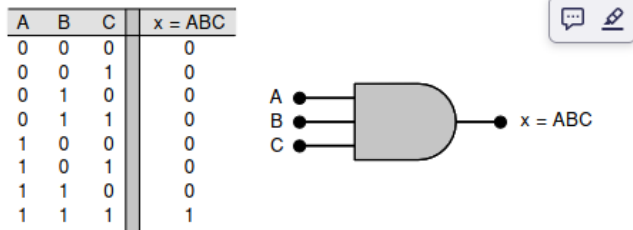


FIGURA 3.8 Tabela-verdade e símbolo para uma porta AND de três entradas.

Exemplo Prático: Operação AND (2 Entradas)

Frase: “Eu só vou à praia se for Sábado **E** se estiver Ensolarado.”

- **A:** É sábado (1 = Sim — 0 = Não)
- **B:** Está ensolarado (1 = Sim — 0 = Não)
- **x:** Vou à praia ($x = A \cdot B$)

A	B	x	Cenário
0	0	0	Não é sábado e chove. (Fico em casa)
0	1	0	Tem sol, mas é dia de semana. (Fico em casa)
1	0	0	É sábado, mas está chovendo. (Fico em casa)
1	1	1	É sábado E tem sol. (Vou à praia!)

Exemplo Prático: Operação AND (3 Entradas)

Frase: “Eu só vou ao parque se for Sábado (A) **E** tiver Sol (B) **E** eu ganhar Carona (C).”

- Expressão:** $x = A \cdot B \cdot C$ (ou $x = ABC$)

A (Sábado)	B (Sol)	C (Carona)	x (Parque)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	0
1	1	1	1

Observação

Diferente do OR, aqui a saída é **0** na maioria dos casos. O resultado só é **1** quando todas as entradas são verdadeiras simultaneamente.

Resumo da Operação AND

- **Aritmética:** A operação AND é realizada da mesma maneira que a multiplicação convencional de 1s e 0s.
- **Circuito:** Uma porta AND é um circuito lógico que realiza fisicamente a operação AND sobre as entradas do circuito.
- **Regra de Saída:** A saída de uma porta AND será **1 somente quando todas as entradas forem 1**; para todos os outros casos, a saída será **0**.
- **Leitura:** A expressão $x = AB$ deve ser lida como: “*x é igual a A e B*”.

Comparação de Saída

Condição de Entrada	Resultado AND
Qualquer entrada é 0	Saída = 0
Todas as entradas são 1	Saída = 1

Diferenciação: Porta AND vs. Porta OR

É fundamental observar a diferença entre os símbolos e comportamentos:

- **Símbolo AND:** Indica que a saída será nível ALTO **somente** quando todas as entradas forem nível ALTO.
- **Símbolo OR:** Indica que a saída será nível ALTO quando **qualquer** entrada for nível ALTO.

Regra Prática

Em um diagrama de circuito, a porta AND exige totalidade de sinais altos, enquanto a porta OR exige apenas uma presença.

Operação NOT (Inversão)

- **Diferencial:** Realizada sobre uma **única variável** de entrada (diferente de OR e AND).
- **Expressão Booleana:** $x = \bar{A}$
- **Leituras:**
 - x é igual a A negado.
 - x é igual ao inverso de A .
 - x é igual ao complemento de A .
- **Conceito:** O valor lógico de saída é o oposto do valor lógico de A .

A	$x = \bar{A}$
0	1
1	0

- $\bar{0} = 1$ (0 é 1 NEGADO).
- $\bar{1} = 0$ (1 é 0 NEGADO).

Indicadores de Inversão:

- Barra sobre a variável ($\bar{A}$).
- Apóstrofo (A').

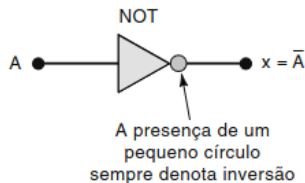
Circuito NOT (INVERSOR)

- **Definição:** Mais comumente denominado **INVERSOR**.
- **Características:**
 - Tem sempre apenas uma entrada.
 - O nível lógico de saída é o oposto ao nível de entrada.
- **Efeito no Sinal:** Inverte (complementa) o sinal de entrada em todos os pontos da forma de onda.
- **Resumo:** Se entrada = 0, saída = 1; se entrada = 1, saída = 0.

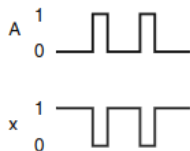
Operacao NOT

NOT		
A		$x = \bar{A}$
0		1
1		0

(a)



(b)



(c)

FIGURA 3.11 (a) Tabela-verdade; (b) símbolo para o INVERSOR (circuito NOT); (c) exemplos de formas de ondas.

Resumo das operações booleanas

As regras para as operações OR, AND e NOT podem ser resumidas como a seguir:

<i>OR</i>	<i>AND</i>	<i>NOT</i>
$0 + 0 = 0$	$0 \cdot 0 = 0$	$\overline{0} = 1$
$0 + 1 = 1$	$0 \cdot 1 = 0$	$\overline{1} = 0$
$1 + 0 = 1$	$1 \cdot 0 = 0$	
$1 + 1 = 1$	$1 \cdot 1 = 1$	

Circuito Exclusive-OR (XOR)

- **Definição:** Circuito que produz uma saída em nível ALTO sempre que as duas entradas estiverem em **níveis opostos**.
- **Abreviação:** XOR.
- **Porta Lógica:** Devido à sua utilidade e frequência de uso, possui um símbolo próprio e é considerada um tipo de porta lógica.
- **Restrição:** Uma porta XOR tem apenas duas entradas; não existem portas XOR de três ou quatro entradas.

- **Forma Abreviada:**

$$x = A \oplus B$$

- **Operação:** O símbolo $\oplus$ representa a operação da porta XOR.

Tabela-Verdade da Porta XOR

A	B	x
0	0	0
0	1	1
1	0	1
1	1	0

Table: Tabela-verdade para $x = A \oplus B$

Características Resumidas da Porta XOR

1. Tem apenas duas entradas.
2. Expressão de saída: $A \oplus B$.
3. A saída será nível ALTO **apenas** quando as duas entradas estiverem em níveis diferentes.

XOR - Ou Exclusivo

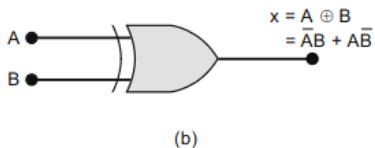


FIGURA 4.20 (a) Circuito XOR e tabela-verdade; (b) símbolo tradicional para a porta XOR.

Porta NOR de Duas Entradas

- É o mesmo que o da porta OR, exceto pelo pequeno círculo na saída que representa a operação de inversão.
- Expressão de saída: $x = \overline{A + B}$
- A porta NOR é equivalente à porta OR seguida de um INVERSOR.
- A saída da porta NOR é exatamente o inverso da saída da porta OR para todas as condições possíveis de entrada.
- Saída da porta OR: nível ALTO quando qualquer entrada for ALTO.
- Saída da porta NOR: nível BAIXO quando qualquer entrada for ALTO.
- Esta operação pode ser estendida para portas NOR com mais de duas entradas.

Porta NOR de Duas Entradas

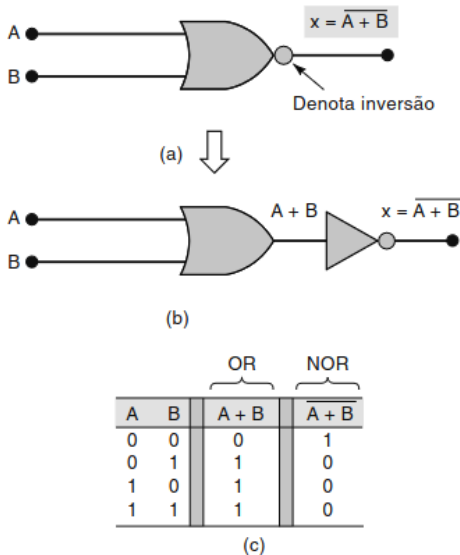


FIGURA 3.19 (a) Símbolo da porta NOR; (b) circuito equivalente; (c) tabela-verdade.

Porta NAND de Duas Entradas

- É o mesmo que o da porta AND, exceto pelo pequeno círculo na saída que indica a operação de inversão.
- Expressão de saída: $x = \overline{AB}$
- A porta NAND é equivalente à porta AND seguida de um INVERSOR.
- A saída da porta NAND é exatamente o inverso da porta AND para todas as condições possíveis de entrada.
- Saída da porta AND: nível ALTO somente quando todas as entradas forem ALTO.
- Saída da porta NAND: nível BAIXO somente quando todas as entradas forem ALTO.
- Esta característica é válida para portas NAND com mais de duas entradas.

Porta NAND de Duas Entradas

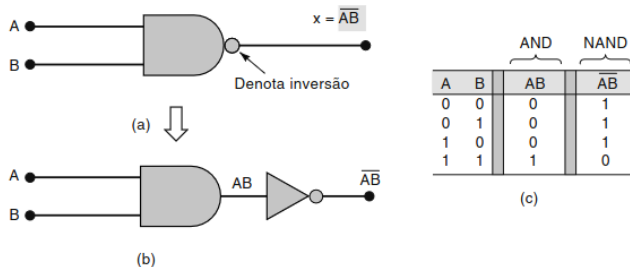


FIGURA 3.22 (a) Símbolo da porta NAND; (b) circuito equivalente; (c) tabela-verdade.

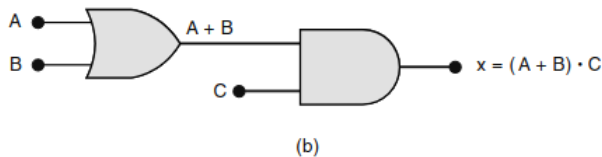
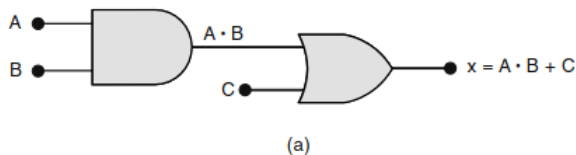
Descrevendo Circuitos Lógicos Algebricamente

- **Princípio Fundamental:** Qualquer circuito lógico, por mais complexo que seja, pode ser descrito usando apenas as três operações booleanas básicas (AND, OR, NOT).
- **Por quê?** Porque as portas AND, OR e INVERSOR são os **blocos fundamentais** (tijolos) de todos os sistemas digitais.
- **Objetivo:** Dado um circuito desenhado, queremos encontrar a expressão algébrica que representa sua saída (xx) em função de suas entradas (A,B,C,... A,B,C,...).

Metodologia

Analisamos o circuito da **entrada para a saída**, escrevendo a expressão booleana na saída de cada porta, sucessivamente, até chegar à saída final.

Exemplo de Análise: Circuito com 3 Entradas



3.13 (a) Circuito lógico e suas expressões booleanas; (b) circuito lógico com expressão que requer parênteses.

Atenção: A Necessidade dos Parênteses

Passo a passo no item b:

1. A porta OR recebe A e B.
2. Sua saída é: **$A+B$**
3. A porta AND recebe $(A + B)$ e a entrada C.
4. A saída final x é o produto lógico (AND) desses valores.
 - Os **parênteses** são essenciais para garantir que a operação OR $(A + B)$ seja realizada **antes** da operação AND com C.
 - Sem os parênteses, a expressão seria interpretada de outra forma.

Precedência de Operadores na Álgebra Booleana

- **O Problema:** A expressão $A \cdot B + C$ pode ser interpretada de **duas maneiras diferentes**:
 - (1) Operação OR de $A \cdot B$ com $C \rightarrow (A \cdot B) + C$
 - (2) Operação AND de A com $B + C \rightarrow A \cdot (B + C)$
- **Resultados diferentes:** Cada interpretação gera uma saída distinta para as mesmas entradas.
- **Solução:** É necessário estabelecer uma **regra de precedência** (ordem das operações).

Regra

Se uma expressão tiver operações AND e OR, a operação **AND** é **realizada primeiro**, a menos que existam **parênteses** indicando o contrário.

Mesma Regra da Álgebra Convencional

- **Comparação:** Esta regra para determinar a ordem das operações é a **mesma** usada na álgebra convencional.

Álgebra Convencional	Álgebra Booleana
Multiplicação antes da Soma $2 + 3 \times 4 = 2 + 12 = 14$	AND antes do OR $A + B \cdot C = A + (B \cdot C)$
Parênteses forçam a Soma primeiro $(2 + 3) \times 4 = 5 \times 4 = 20$	Parênteses forçam o OR primeiro $(A + B) \cdot C$

- **Conclusão:** A precedência do operador AND (multiplicação lógica) sobre o OR (soma lógica) segue a mesma lógica intuitiva da matemática tradicional.

Exemplo Prático: Uso dos Parênteses

Na figura 3.13 (b)

Análise do Circuito:

1. A porta OR recebe A e B $\rightarrow$ saída = $A + B$
2. Esta saída vai para uma porta AND junto com a entrada C.
3. Portanto: $x = (A + B) \cdot C$

Por que os parênteses são necessários?

Sem os parênteses, a expressão $A + B \cdot C$ seria interpretada como $A + (B \cdot C)$ — um circuito completamente diferente!

O Que Acontece Sem os Parênteses?

- A expressão $A + B \cdot C$ (sem parênteses) significa, pela regra de precedência:

$$x = A + (B \cdot C)$$

- Primeiro calcula-se $B \cdot C$ (AND), depois faz-se OR com A.
- Este é um **circuito diferente** do original da Figura 3.13(b).

Cuidado!

Os parênteses não são opcionais. Eles alteram completamente a expressão lógica e, conseqüentemente, o circuito resultante.

Circuitos com INVERSORES Lógicos

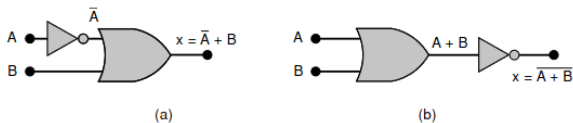


FIGURA 3.14 Circuitos com INVERSORES.

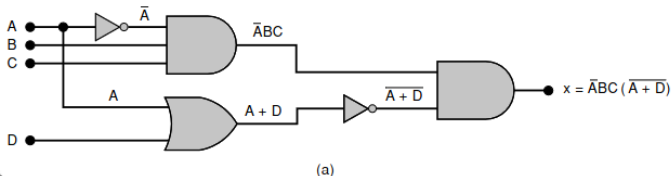


FIGURA 3.15 Mais exemplos (*continua*).

Circuitos com INVERSORES Lógicos

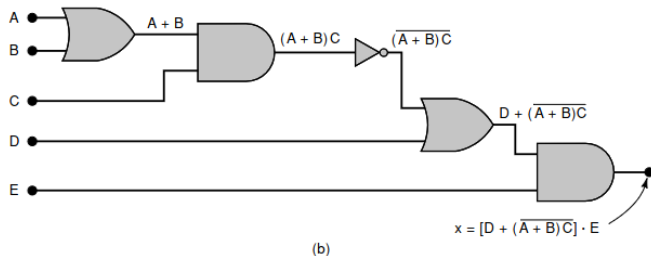
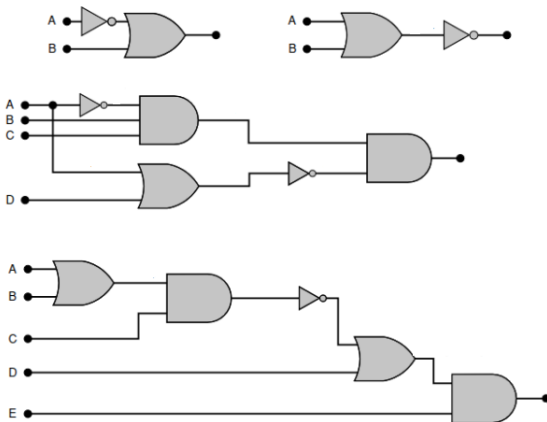
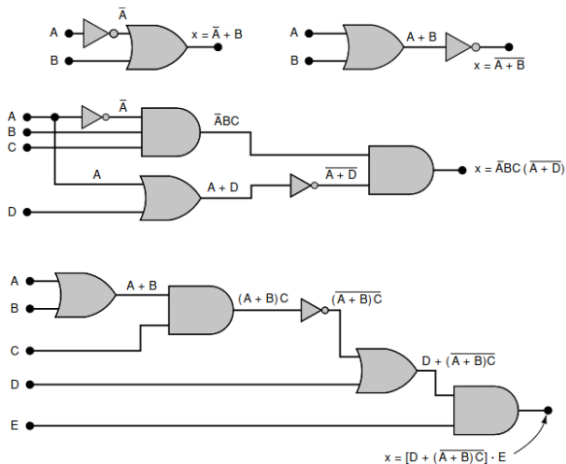


FIGURA 3.15 Mais exemplos (*continuação*).

Exercício - Faça as expressões dos circuitos



Exercício (GABARITO)- Faça as expressões dos circuitos



Circuitos com INVERSORES Lógicos

- Sempre que um INVERSOR estiver presente, a expressão para sua saída será igual à expressão de entrada com uma barra sobre ela.
- **Figura 3.14(a):** A entrada A passa por um INVERSOR (saída = $\bar{A}$), que alimenta uma porta OR com B $\rightarrow x = \bar{A} + B$
- **Figura 3.14(b):** A porta OR produz $A + B$, que entra em um INVERSOR $\rightarrow x = \overline{A + B}$

Exercício: faça a tabela verdade das expressões abaixo

- $\overline{A + B}$
- $\bar{A} + \bar{B}$

VEJA que as expressões $\overline{A + B}$ e $\bar{A} + \bar{B}$ **não são equivalentes!**

Regras para Avaliar uma Expressão Booleana

1. Primeiro, realize as inversões de termos simples; ou seja, $\bar{0} = 1$ ou $\bar{1} = 0$.
2. Em seguida, realize as operações dentro de parênteses.
3. Realize as operações AND antes das operações OR, a menos que os parênteses indiquem o contrário.
4. Se uma expressão tiver uma barra sobre, realize a operação indicada pela expressão e, em seguida, inverta o resultado.

Obtendo o Nível Lógico de Saída

- De posse da expressão booleana para a saída de um circuito, podemos obter o nível lógico da saída para qualquer conjunto de níveis lógicos de entrada.
- **Exemplo:** Determine o nível lógico da saída x para o circuito da Figura 3.15(a) com $A = 0$, $B = 1$, $C = 1$ e $D = 1$.
- **Procedimento:** Assim como na álgebra convencional, substituímos os valores das variáveis na expressão e realizamos as operações indicadas.

$$\begin{aligned}x &= \overline{A}BC(\overline{A + D}) \\&= \overline{0} \cdot 1 \cdot 1 \cdot (\overline{0 + 1}) \\&= 1 \cdot 1 \cdot 1 \cdot (\overline{0 + 1}) \\&= 1 \cdot 1 \cdot 1 \cdot (\overline{1}) \\&= 1 \cdot 1 \cdot 1 \cdot 0 \\&= 0\end{aligned}$$

Segundo Exemplo de Substituição

- Vamos determinar a saída do circuito da Figura 3.15(b) para:
- $A = 0$, $B = 0$, $C = 1$, $D = 1$ e $E = 1$

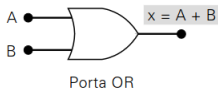
$$\begin{aligned}x &= [D + \overline{(A + B)C}] \cdot E \\&= [1 + \overline{(0 + 0) \cdot 1}] \cdot 1 \\&= [1 + \overline{0 \cdot 1}] \cdot 1 \\&= [1 + \overline{0}] \cdot 1 \\&= [1 + 1] \cdot 1 \\&= 1 \cdot 1 \\&= 1\end{aligned}$$

Exercício

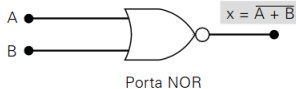
Determine as saídas dos dois circuitos na Figura 3.15 no caso em que todas as entradas forem 1. As respostas são $x = 0$ e $x = 1$, respectivamente.

Resumo: Portas lógicas

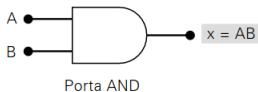
SÍMBOLOS DAS PORTAS LÓGICAS



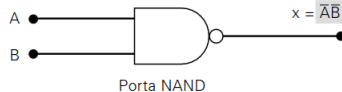
Porta OR



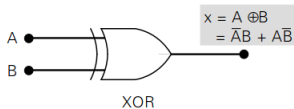
Porta NOR



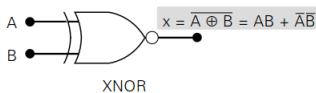
Porta AND



Porta NAND



XOR



XNOR

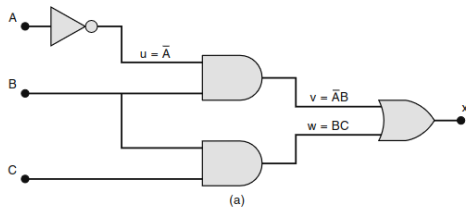
Análise utilizando uma Tabela-Verdade

- Quando se tem um circuito lógico combinacional e se deseja saber como ele funciona, podemos usar uma tabela-verdade.
- **Este método:**
 - Permite que se analise uma porta ou combinação lógica de cada vez.
 - Permite que se confira facilmente o trabalho.
 - Quando o trabalho se encerra, há uma tabela que ajuda a verificação de erros do circuito lógico.

Análise utilizando uma Tabela-Verdade

- Lembre-se de que uma tabela-verdade lista as possíveis combinações de entrada em ordem numérica.
- Para cada possível combinação de entrada, podemos determinar o estado lógico em cada ponto (nó) do circuito lógico, inclusive a saída.
- Exemplo: Na Figura 3.16(a), há vários nós intermediários (conexões entre a saída de uma porta e a entrada de outra), chamados de u , v e w .
- O primeiro passo, após listar todas as combinações de entrada, é criar uma coluna na tabela-verdade para cada sinal intermediário (nó).

Circuito com tabela verdade



A	B	C	$u = \bar{A}$	$v = \bar{A}B$	$w = BC$	$x = v + w$
0	0	0	1			
0	0	1	1			
0	1	0	1			
0	1	1	1			
1	0	0	0			
1	0	1	0			
1	1	0	0			
1	1	1	0			

(b)

A	B	C	$u = \bar{A}$	$v = \bar{A}B$	$w = BC$	$x = v + w$
0	0	0	1	0		
0	0	1	1	0		
0	1	0	1	1		
0	1	1	1	1		
1	0	0	0	0		
1	0	1	0	0		
1	1	0	0	0		
1	1	1	0	0		

(c)

A	B	C	$u = \bar{A}$	$v = \bar{A}B$	$w = BC$	$x = v + w$
0	0	0	1	0	0	
0	0	1	1	0	0	
0	1	0	1	1	0	
0	1	1	1	1	1	
1	0	0	0	0	0	
1	0	1	0	0	0	
1	1	0	0	0	0	
1	1	1	0	0	1	

(d)

A	B	C	$u = \bar{A}$	$v = \bar{A}B$	$w = BC$	$x = v + w$
0	0	0	1	0	0	0
0	0	1	1	0	0	0
0	1	0	1	1	0	1
0	1	1	1	1	1	1
1	0	0	0	0	0	0
1	0	1	0	0	0	0
1	1	0	0	0	0	0
1	1	1	0	0	1	1

(e)

Exercício

Analise a operação da Figura 3.15(a) criando uma tabela que mostre o estado lógico em cada nó do circuito.

Exercício (GABARITO)

Exemplo 3.6

Analise a operação da Figura 3.15(a) criando uma tabela que mostre o estado lógico em cada nó do circuito.

Solução

Preencha a coluna t com 1s sempre que $A = 0$ e $B = 1$ e $C = 1$.

Preencha a coluna u com 1s sempre que $A = 1$ ou $D = 1$.

Preencha a coluna v com o complemento de todas as linhas da coluna u .

Preencha a coluna x com 1s sempre que $t = 1$ e $v = 1$.

A	B	C	D	$t = \overline{A}BC$	$u = A + D$	$v = \overline{A + D}$	$x = tv$
0	0	0	0	0	0	1	0
0	0	0	1	0	1	0	0
0	0	1	0	0	0	1	0
0	0	1	1	0	1	0	0
0	1	0	0	0	0	1	0
0	1	0	1	0	1	0	0
0	1	1	0	1	0	1	1
0	1	1	1	1	1	0	0
1	0	0	0	0	1	0	0
1	0	0	1	0	1	0	0
1	0	1	0	0	1	0	0
1	0	1	1	0	1	0	0
1	1	0	0	0	1	0	0
1	1	0	1	0	1	0	0
1	1	1	0	0	1	0	0
1	1	1	1	0	1	0	0

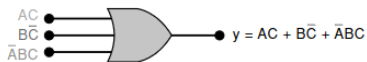
TABELAS-VERDADE DAS PORTAS LÓGICAS

A	B	OR $A + B$	NOR $\overline{A + B}$	AND $A \cdot B$	NAND $\overline{A \cdot B}$	XOR $A \oplus B$	XNOR $\overline{A \oplus B}$
0	0	0	1	0	1	0	1
0	1	1	0	0	1	1	0
1	0	1	0	0	1	1	0
1	1	1	0	1	0	0	1

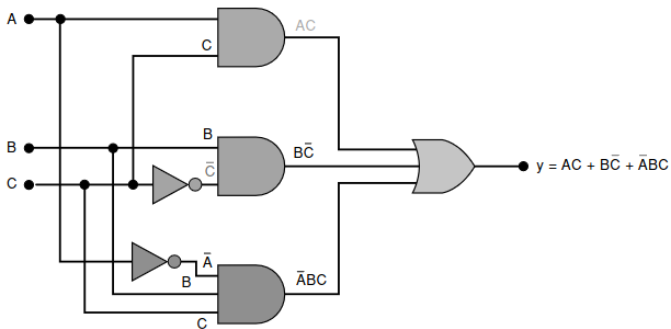
Implementando circuitos a partir de expressões booleanas

- Quando a operação de um circuito é definida por uma expressão booleana, podemos desenhar o diagrama do circuito lógico diretamente a partir da expressão.
- **Exemplos:**
 - Se precisarmos de um circuito definido por $x = A \cdot B \cdot C \rightarrow$ porta AND de três entradas.
 - Se precisarmos de um circuito definido por $x = \bar{A} + B \rightarrow$ porta OR de duas entradas com um INVERSOR em uma das entradas.
- O mesmo raciocínio pode ser estendido para circuitos mais complexos.

Exemplo de Implementação



(a)



(b)

FIGURA 3.17 Construindo um circuito lógico a partir de uma expressão booleana.

Exercício

Desenhe o diagrama do circuito que implemente a expressão $x = (A + B)(\bar{B} + C)$

Exercício (GABARITO)

Exemplo 3.7

Desenhe o diagrama do circuito que implemente a expressão $x = (A + B)(\bar{B} + C)$.

Solução

Essa expressão mostra que os termos $A + B$ e $\bar{B} + C$ são entradas de uma porta AND, e cada um deles é gerado por portas OR independentes. O resultado é demonstrado na Figura 3.18.

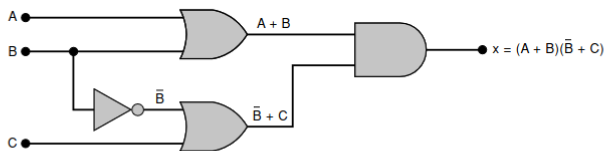


FIGURA 3.18 Exemplo 3.7.



Teoremas Booleanos - uma única variável

Ler no livro - Pag. 67.

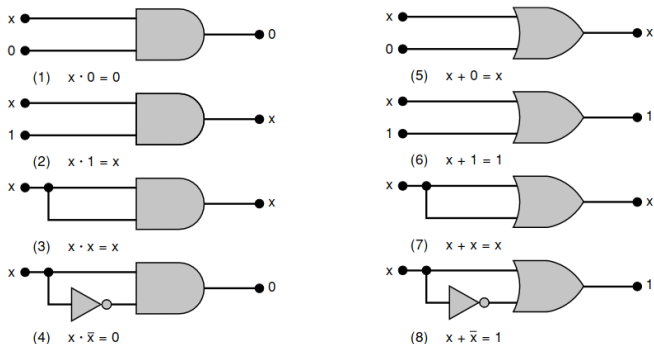


FIGURA 3.25 Teoremas para uma única variável.

Teoremas Booleanos - com mais de uma variável

Ler no livro - Pag. 68.

$$(9) \quad x + y = y + x$$

$$(10) \quad x \cdot y = y \cdot x$$

$$(11) \quad x + (y + z) = (x + y) + z = x + y + z$$

$$(12) \quad x(yz) = (xy)z = xyz$$

$$(13a) \quad x(y + z) = xy + xz$$

$$(13b) \quad (w + x)(y + z) = wy + xy + wz + xz$$

$$(14) \quad x + xy = x$$

$$(15a) \quad x + \bar{x}y = x + y$$

$$(15b) \quad \bar{x} + xy = \bar{x} + y$$

Teoremas Booleanos - teoremas de DeMorgan

Ler no livro - Pag. 70.

Os teoremas de DeMorgan são muito úteis na simplificação de expressões nas quais um produto ou uma soma de variáveis aparecem negados (barrados).

$$(16) \quad \overline{(x + y)} = \bar{x} \cdot \bar{y}$$

$$(17) \quad \overline{(x \cdot y)} = \bar{x} + \bar{y}$$

Os teoremas de DeMorgan são facilmente estendidos para mais de duas variáveis.

$$\begin{aligned}\overline{x + y + z} &= \bar{x} \cdot \bar{y} \cdot \bar{z} \\ \overline{x \cdot y \cdot z} &= \bar{x} + \bar{y} + \bar{z}\end{aligned}$$

TEOREMAS BOOLEANOS

- | | | |
|---|---|--|
| 1. $x \cdot 0 = 0$ | 2. $x \cdot 1 = x$ | 3. $x \cdot x = x$ |
| 4. $x \cdot \bar{x} = 0$ | 5. $x + 0 = x$ | 6. $x + 1 = 1$ |
| 7. $x + x = x$ | 8. $x + \bar{x} = 1$ | 9. $x + y = y + x$ |
| 10. $x \cdot y = y \cdot x$ | 11. $x + (y + z) = (x + y) + z = x + y + z$ | 12. $x(yz) = (xy)z = xyz$ |
| 13a. $x(y + z) = xy + xz$ | 13b. $(w + x)(y + z) = wy + xy + wz + xz$ | 14. $x + xy = x$ |
| 15a. $x + \bar{x}y = x + y$ | 15b. $\bar{x} + xy = \bar{x} + y$ | 16. $\overline{x + y} = \bar{x} \bar{y}$ |
| 17. $\overline{xy} = \bar{x} + \bar{y}$ | | |

Exercício de Simplificação 1

Questão

Simplifique a expressão:

$$y = A\bar{B}D + A\bar{B}\bar{D}$$

- Aplique os teoremas da álgebra booleana para simplificar a expressão.

Exercício de Simplificação 1 (Gabarito)

Exemplo 3.13

Simplifique a expressão $y = A\bar{B}D + A\bar{B}\bar{D}$.

Solução

Colocando-se em evidência as variáveis comuns, $A\bar{B}$, usando o teorema (13), temos:

$$y = A\bar{B}(D + \bar{D})$$

Usando-se o teorema (8), o termo entre parênteses é equivalente a 1. Assim,

$$\begin{aligned} y &= A\bar{B} \cdot 1 \\ &= A\bar{B} \quad [\text{usando o teorema (2)}] \end{aligned}$$

Questão

Simplifique a expressão:

$$z = (\bar{A} + B)(A + B)$$

- Aplique os teoremas da álgebra booleana para simplificar a expressão.

Exercício de Simplificação 2 (Gabarito)

Exemplo 3.14

Simplifique $z = (\bar{A} + B)(A + B)$.

Solução

A expressão pode ser expandida, multiplicando-se os termos [teorema (13)]:

$$z = \bar{A} \cdot A + \bar{A} \cdot B + B \cdot A + B \cdot B$$

Aplicando-se o teorema (4), o termo $\bar{A} \cdot A = 0$. Além disso, $B \cdot B = B$ [teorema (3)]:

$$z = 0 + \bar{A} \cdot B + B \cdot A + B = \bar{A}B + AB + B$$

Colocando-se em evidência a variável B [teorema (13)], temos:

$$z = B(\bar{A} + A + 1)$$

Finalmente, usando-se os teoremas (2) e (6), temos:

$$z = B$$

Questão

Simplifique a expressão:

$$x = ACD + \bar{A}BCD$$

- Aplique os teoremas da álgebra booleana para simplificar a expressão.

Exercício de Simplificação 3 (Gabarito)

Exemplo 3.15

Simplifique $x = ACD + \bar{A}BCD$.

Solução

Colocando-se em evidência os termos comuns CD , temos:

$$x = CD(A + \bar{A}B)$$

Usando-se o teorema (15a), podemos substituir $A + \bar{A}B$ por $A + B$. Assim,

$$\begin{aligned}x &= CD(A + B) \\ &= ACD + BCD\end{aligned}$$

Simplifique a expressão $z = \overline{(\overline{A} + C) \cdot (B + \overline{D})}$

Exercício de Simplificação 4 (Gabarito)

Exemplo 3.16

Simplifique a expressão $z = \overline{(\bar{A} + C)} \cdot (B + \bar{D})$ para que tenha apenas variáveis simples invertidas.

Solução

Usando o teorema (17) e considerando $(\bar{A} + C)$ como x e $(B + \bar{D})$ como y , temos:

$$z = \overline{(\bar{A} + C)} + \overline{(B + \bar{D})}$$

Podemos pensar sobre essa operação como a quebra de uma barra grande ao meio e a troca do sinal AND ($\cdot$) pelo sinal OR (+). Agora, o termo $\overline{(\bar{A} + C)}$ pode ser simplificado aplicando o teorema (16). Do mesmo modo, $\overline{(B + \bar{D})}$ também pode ser simplificado:

$$\begin{aligned} z &= \overline{(\bar{A} + C)} + \overline{(B + \bar{D})} \\ &= (\bar{\bar{A}} \cdot \bar{C}) + \bar{B} \cdot \bar{\bar{D}} \end{aligned}$$

Nesse caso, partimos a barra grande ao meio e substituímos o sinal (+) por ($\cdot$). Cancelando as duplas inversões, temos:

$$z = A\bar{C} + \bar{B}D$$

Exercício de Simplificação 5 (Gabarito)

Exemplo 1

$$\begin{aligned} z &= \overline{A + \overline{B} \cdot C} \\ &= \overline{A} \cdot \overline{(\overline{B} \cdot C)} \\ &= \overline{A} \cdot (\overline{\overline{B}} + \overline{C}) \\ &= \overline{A} \cdot (B + \overline{C}) \end{aligned}$$

Exemplo 2

$$\begin{aligned} \omega &= \overline{(A + BC) \cdot (D + EF)} \\ &= \overline{(A + BC)} + \overline{(D + EF)} \\ &= (\overline{A} \cdot \overline{BC}) + (\overline{D} \cdot \overline{EF}) \\ &= [\overline{A} \cdot (\overline{B} + \overline{C})] + [\overline{D} \cdot (\overline{E} + \overline{F})] \\ &= \overline{A} \overline{B} + \overline{A} \overline{C} + \overline{D} \overline{E} + \overline{D} \overline{F} \end{aligned}$$

Exercício de Simplificação 6 (Gabarito)

Exemplo 3.17

Determine a expressão de saída para o circuito da Figura 3.28 e simplifique-a usando os teoremas de DeMorgan.

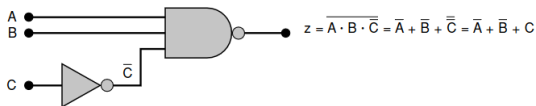


FIGURA 3.28 Exemplo 3.17.

Solução

A expressão para z é $z = A\bar{B}\bar{C}$. Usando-se o teorema de DeMorgan para partir a barra maior, temos:

$$z = \overline{\overline{A} + \overline{B} + \overline{\bar{C}}}$$

Cancelando-se a dupla inversão sobre a variável C , obtemos:

$$z = \overline{\overline{A} + \overline{B} + C}$$

Aula 8 - Circuitos e Unidade Lógica e Aritmética

Prof. Gabriel Rodrigues Caldas de Aquino

Instituto de Computação
Universidade Federal do Rio de Janeiro
gabrielaquino@ic.ufrj.br

Compilado em:
May 19, 2026

Circuito Somador

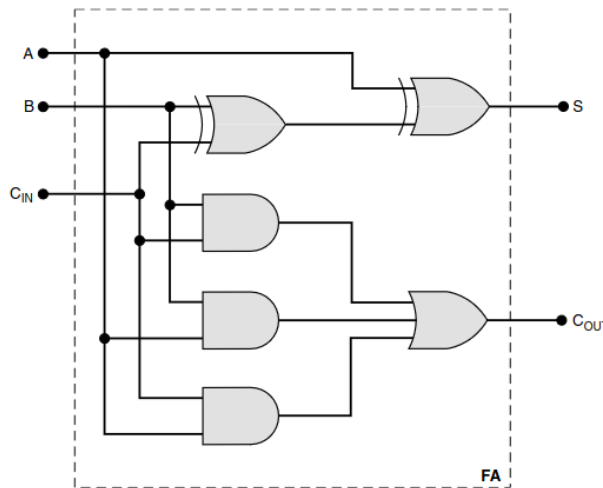


FIGURA 6.8 Circuito para um somador completo.

Circuito Somador - Tabela verdade

Entradas de bits da primeira parcela	Entradas de bits da segunda parcela	Entradas de bits do carry	Saída de bits da soma	Saída de bits do carry
A	B	C_{IN}	S	C_{OUT}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

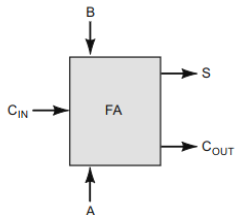


FIGURA 6.7 Tabela-verdade para um circuito somador completo.

Unidade Lógica e aritmética

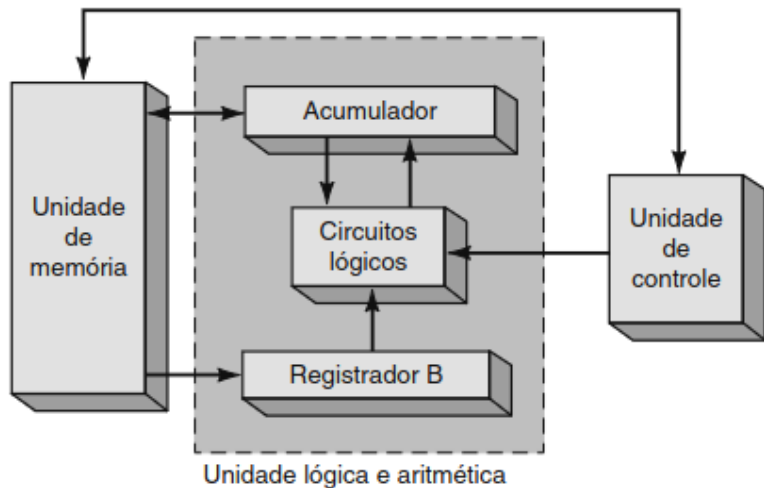


FIGURA 6.4 Blocos funcionais de uma ALU.

A Unidade Lógica e Aritmética (ULA / ALU)

- Todas as operações lógicas e aritméticas são realizadas na **unidade lógica e aritmética** (ALU) de um computador.
- **Objetivo principal:** Receber dados binários armazenados na memória e executar operações sobre eles, de acordo com instruções da unidade de controle.

Estrutura Básica

Contém pelo menos dois registradores (**Registrador B** e **Acumulador**) e uma **lógica combinacional** que realiza as operações sobre os números armazenados neles.

Fluxo Típico de Operações

1. **Instrução:** A unidade de controle decodifica a instrução indicando que um número de uma posição da memória deve ser somado ao **acumulador**.
2. **Carga:** O número a ser somado é transferido da memória para o **registrador B**.
3. **Processamento:** Os números de B e do acumulador são somados no circuito lógico. O resultado é enviado e **sobrescreve o acumulador**.
4. **Destino:** O novo valor no acumulador pode ser mantido para sucessivas somas ou transferido de volta para a memória.

O Papel do Acumulador

Ele "*acumula*" os resultados dos passos intermediários de problemas aritméticos que implicam múltiplos passos, além do resultado final.

As Quatro Operações na ALU

Apesar de a ALU ser baseada em circuitos somadores, ela consegue realizar todas as operações aritméticas fundamentais:

- **Soma:** Executada diretamente pelos circuitos somadores binários.
- **Subtração:** Realizada através da soma com o **complemento de dois** ($A - B = A + [B_{comp2}]$).
- **Multiplicação:** Implementada como uma sucessão de somas e deslocamentos de bits (*shifts*).
- **Divisão:** Processada através de subtrações sucessivas e comparações.

Divisão por Subtrações Sucessivas ($27 \div 5$)

Conceito: A ALU realiza a divisão subtraindo o divisor do dividendo repetidamente até que o resultado seja menor que o divisor.

- **Dividendo:** 27
- **Divisor:** 5

Processo Iterativo

1. $27 - 5 = 22$ (1ª subtração)
2. $22 - 5 = 17$ (2ª subtração)
3. $17 - 5 = 12$ (3ª subtração)
4. $12 - 5 = 7$ (4ª subtração)
5. $7 - 5 = 2$ (5ª subtração)

Resultado Final

- **Quociente:** 5
(Número de subtrações realizadas)
- **Resto:** 2
(Valor final menor que o divisor)

Lógica da ALU:

Contador de iterações =
Quociente
Resíduo final = Resto

A Máquina IAS

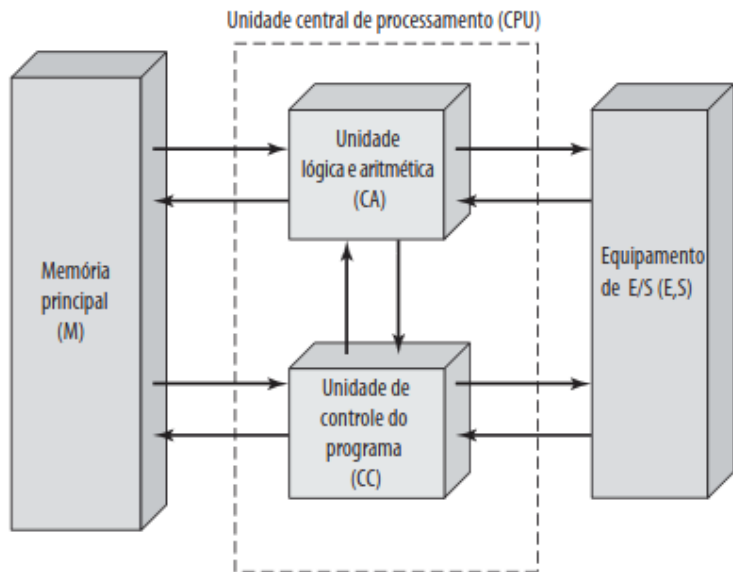
- **Origem:** Foi o primeiro computador eletrônico construído no *Institute for Advanced Study* (IAS) em Princeton, Nova Jersey.
- **A Máquina de von Neumann:** Recebe frequentemente este nome porque o artigo que descrevia o seu projeto foi editado por **John von Neumann**, professor de matemática da Universidade de Princeton e do IAS.
- **Linha do Tempo:** Construído sob sua direção, o projeto teve início em **1946** e foi concluído em **1951**.
- [Veja o link na Wikipédia](#)

Legado da Arquitetura

A organização geral desenvolvida passou a ser chamada de **Arquitetura de von Neumann**, embora tenha sido concebida e implementada também por outros pesquisadores do instituto.

Presente hoje na coleção do *Smithsonian National Museum of American History*.

Estrutura básica do Computador IAS

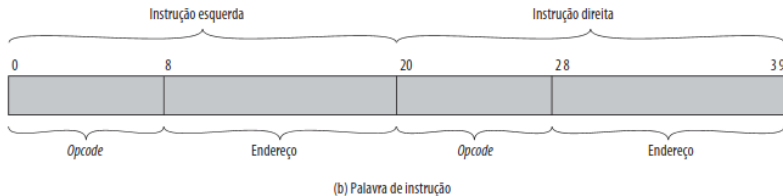
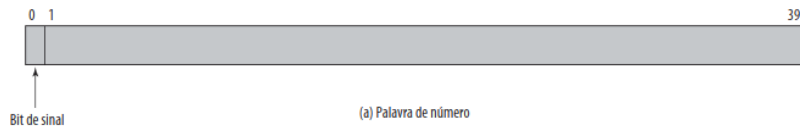


A Estrutura Geral do Computador IAS

O computador IAS foi baseado na proposta inicial de von Neumann e serve como base para os computadores modernos. Ele consiste em quatro componentes fundamentais:

- **Memória Principal (M):** Responsável por armazenar tanto os dados quanto as instruções do programa.
- **Unidade Lógica e Aritmética (CA - Central Arithmetic):** Unidade especializada para realizar as operações aritméticas elementares e lógicas.
- **Unidade de Controle (CC - Central Control):** Órgão central que decodifica as instruções e sequencia a execução das operações.
- **Entrada e Saída (I/O - Input/Output):** Unidades encarregadas de realizar o contato com o meio exterior de gravação (R).

Estrutura da memória



Formatos de Palavra no Computador IAS

A memória do IAS é organizada em 1.000 locais de armazenamento chamados **palavras** (*words*), onde cada palavra possui **40 bits**.

Representação de Dados

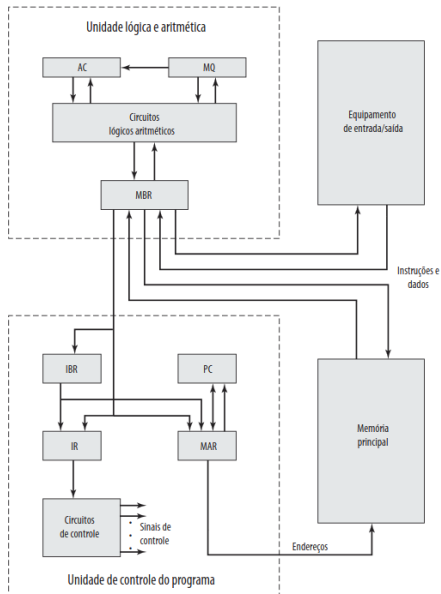
- **1 bit de sinal** seguido por um valor de **39 bits**.

Representação de Instruções

Uma palavra pode armazenar até **duas instruções de 20 bits** cada. Cada instrução é dividida em:

- **Código de Operação (Opcode):** 8 bits para especificar a operação.
- **Endereço:** 12 bits para apontar para uma das 1.000 palavras na memória.

Computador IAS mais a fundo



Registradores da Unidade de Controle e ULA

Para buscar e executar as instruções passo a passo, o IAS utiliza um conjunto específico de registradores internos:

- **mBR (Memory Buffer Register):** Contém a palavra que será enviada para a memória/E/S ou que acabou de ser recebida deles.
- **mAR (Memory Address Register):** Especifica o endereço de memória da palavra que será lida ou gravada via mBR.
- **IR (Instruction Register):** Guarda o opcode de 8 bits da instrução que está sendo executada no momento.
- **IBR (Instruction Buffer Register):** Armazena temporariamente a segunda instrução da palavra para ser executada a seguir.
- **PC (Program Counter):** Contém o endereço do próximo par de instruções que deve ser buscado na memória.

Para lidar com operandos e resultados da ALU, o IAS utiliza dois registradores especiais que trabalham em conjunto:

- **AC (Acumulador) e MQ (Multiplier Quotient):** Empregados para manter temporariamente os operandos e os resultados das operações.

Multiplicação de 40 bits → Resultado de 80 bits

Quando a ALU multiplica dois números de 40 bits, o resultado de 80 bits é dividido:

- **AC:** Armazena os 40 bits **mais significativos**.
- **MQ:** Armazena os 40 bits **menos significativos**.

O Ciclo de Instrução do IAS

O funcionamento do IAS baseia-se na repetição contínua de um ciclo de instrução, dividido em dois subciclos principais:

1. Ciclo de Busca (Fetch Cycle):

- O *opcode* da próxima instrução é carregado no **IR**.
- A parte do endereço é carregada no **MAR**.
- **Origem:** A instrução pode vir direto do **IBR** (se já estiver lá) ou ser buscada na memória via **MBR**.

2. Ciclo de Execução (Execute Cycle):

- Com o *opcode* no **IR**, o circuito de controle o interpreta.
- Sinais de controle são enviados para mover dados ou ativar funções da ALU.

Por que a Indireção de Registradores?

No IAS, apenas o **MAR** especifica endereços de leitura/escrita e apenas o **MBR** serve como origem/destino de dados da memória.

Justificativa de Hardware

- **Simplificação Eletrônica:** Reduz drasticamente a quantidade de caminhos de dados (*data paths*) e multiplexadores necessários.
- **Controle Centralizado:** Facilita o projeto dos circuitos lógicos da Unidade de Controle.

O Conjunto de Instruções do IAS (21 Instruções)

As 21 instruções do computador IAS podem ser agrupadas em 5 categorias funcionais:

- **Transferência de Dados:** Movem dados entre a memória e registradores da ALU, ou entre os próprios registradores da ALU.
- **Desvio Incondicional:** Altera a execução sequencial do programa, permitindo a criação de laços (*loops*).
- **Desvio Condicional:** Permite tomar decisões no código baseando-se em condições (ex: se o valor for positivo).
- **Aritméticas:** Operações matemáticas executadas diretamente pelos circuitos da ALU.
- **Modificação de Endereço:** Permite que endereços sejam calculados na ALU e inseridos em instruções na memória, garantindo alta flexibilidade de endereçamento.

O Conjunto de Instruções do IAS (21 Instruções)

Tabela 2.1 O conjunto de instruções do IAS

Tipo de instrução	Opcode	Representação simbólica	Descrição
Transferência de dados	00001010	LOAD MQ	Transfere o conteúdo de MQ para AC
	00001001	LOAD MQ,M(X)	Transfere o conteúdo do local de memória X para MQ
	00100001	STOR M(X)	Transfere o conteúdo de AC para o local de memória X
	00000001	LOAD M(X)	Transfere M(X) para o AC
	00000010	LOAD - M(X)	Transfere - M(X) para o AC
	00000011	LOAD M(X)	Transfere o valor absoluto de M(X) para o AC
	00000100	LOAD - M(X)	Transfere - M(X) para o acumulador
Desvio incondicional	00001101	JUMP M(X,0:19)	Apanha a próxima instrução da metade esquerda de M(X)
	00001110	JUMP M(X,20:39)	Apanha a próxima instrução da metade direita de M(X)
Desvio condicional	00001111	JUMP+ M(X,0:19)	Se o número no AC for não negativo, apanha a próxima instrução da metade esquerda de M(X)
	00010000	JUMP+ M(X,20:39)	Se o número no AC for não negativo, apanha a próxima instrução da metade direita de M(X)
Aritmética	00000101	ADD M(X)	Soma M(X) a AC; coloca o resultado em AC
	00000111	ADD M(X)	Soma M(X) a AC; coloca o resultado em AC
	00000110	SUB M(X)	Subtrai M(X) de AC; coloca o resultado em AC
	00001000	SUB M(X)	Subtrai M(X) de AC; coloca o resto em AC
	00001011	MUL M(X)	Multiplica M(X) por MQ; coloca os bits mais significativos do resultado em AC; coloca bits menos significativos em MQ
	00001100	DIV M(X)	Divide AC por M(X); coloca o quociente em MQ e o resto em AC
	00010100	LSH	Multiplica o AC por 2; ou seja, desloca à esquerda uma posição de bit
	00010101	RSH	Divide o AC por 2; ou seja, desloca uma posição à direita
Modificação de endereço	00010010	STOR M(X,8:19)	Substitui campo de endereço da esquerda em M(X) por 12 bits mais à direita de AC
	00010011	STOR M(X,28:39)	Substitui campo de endereço da direita em M(X) por 12 bits mais à direita de AC